

Nutrition Assistant (MERN)

Full Stack Development (FSD) Documentation

INTRODUCTION

Nutrition Assistant is a user-friendly web app built for health-conscious individuals, students, and working professionals who want a simple way to track what they eat. The platform provides food search and nutritional lookup, daily calorie tracking, and macronutrient analysis, powered by a searchable food/nutrition database, making it easier for users to make informed dietary decisions. With a simple registration and secure login process, users can access their personalized dashboard to log meals, track daily calorie and macronutrient totals, and review their progress over time. Whether someone is just starting to pay attention to nutrition or already experienced at tracking macros, Nutrition Assistant offers a seamless, low-friction experience. By combining a clean interface with reliable nutrition data and simple tracking tools, it ensures that every logging session is quick and every insight is clear, helping users build healthier habits with confidence.

Scenario based case study

Scenario

Priya is a working professional who wants to eat healthier but struggles to find time for manual nutrition tracking. She wants to understand what she is eating and stay within a reasonable daily calorie range without spending hours on spreadsheets. Priya decides to use Nutrition Assistant, a MERN-based nutrition tracking application, to log her meals and monitor her progress.

User Registration and Profile Creation

Priya visits the Nutrition Assistant website and completes the registration process. She provides her name, email, and a password to create her account. Her registration is successful, and she is redirected to the Login page.

Login and Dashboard Access

Priya securely logs in using her registered email and password. Upon successful login, she gains access to her personalized dashboard, where she can see her nutrition summary for the day and quickly jump into logging a meal.

Explore Food Listings

Priya navigates to the “Food Search” section to look up items she plans to eat. She can search by name and instantly see calories, protein, carbohydrates, and fat for each item.

Logging Daily Intake

Priya selects a food item, enters the quantity she ate, and logs it to her daily record. Her calorie and macronutrient totals update immediately, giving her real-time feedback on her progress toward her goals.

Progress Tracking and Analysis

Priya monitors her nutrition dashboard throughout the day, checking her running calorie total and macronutrient breakdown to decide what to eat next and stay on track.

Building Better Habits

Over time, Priya builds a consistent logging habit. The simplicity of the dashboard means she spends seconds, not minutes, keeping her nutrition data up to date — helping her stay motivated and informed.

Continuous Improvement

The Nutrition Assistant development team continuously improves the platform based on user feedback, adding new features, enhancing performance, and ensuring reliability and security, so users like Priya keep getting a seamless, trustworthy tracking experience.

SYSTEM REQUIREMENTS

To ensure smooth development, deployment, and usage of the Nutrition Assistant web application, certain system prerequisites must be met. These requirements are categorized into software and hardware specifications, all of which contribute to building a robust and scalable nutrition tracking platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

Operating System	Windows 10/11, macOS, or Linux — supports cross-platform development and testing.
Node.js (v18 or above)	Provides the JavaScript runtime for both the React build tooling and the Express server.
npm (v9 or above)	Package manager required to install dependencies for both client and server.
React.js	JavaScript library for building the dynamic, responsive user interface.
Browser	Google Chrome / Firefox (latest version) — for rendering and testing the UI in real time.
Express.js	Lightweight web framework for building RESTful APIs.
MongoDB (Atlas or local)	NoSQL database used to store users, food items, and nutrition records.
Postman	Tool for testing APIs during development.
Visual Studio Code	Preferred code editor with built-in Git and terminal support.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support development and usage of the application.

Processor	Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — ensures fast compilation and multitasking during development.
RAM	Minimum 8 GB (16 GB recommended) — for running development servers, IDE, and browser simultaneously.
Storage	At least 1 GB free space — required for package installations and local project files.
Display	1366x768 or higher — recommended for a comfortable coding and layout-testing experience.

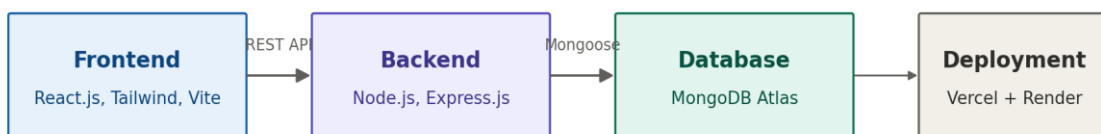
PROJECT ARCHITECTURE

Nutrition Assistant follows a modular full-stack architecture using the MERN stack:

Frontend (React) <==> Backend (Express + Node.js) <==> Database (MongoDB Atlas)

- Frontend: Built using React, Tailwind CSS, and Vite, it provides the user interface for registration, login, food search, calorie tracking, and the nutrition dashboard.
- Backend: Built using Node.js and Express, it handles API requests, user authentication, food search, and food-logging business logic.
- Database: MongoDB Atlas is used to store data such as users, food items, and nutrition (intake) records.

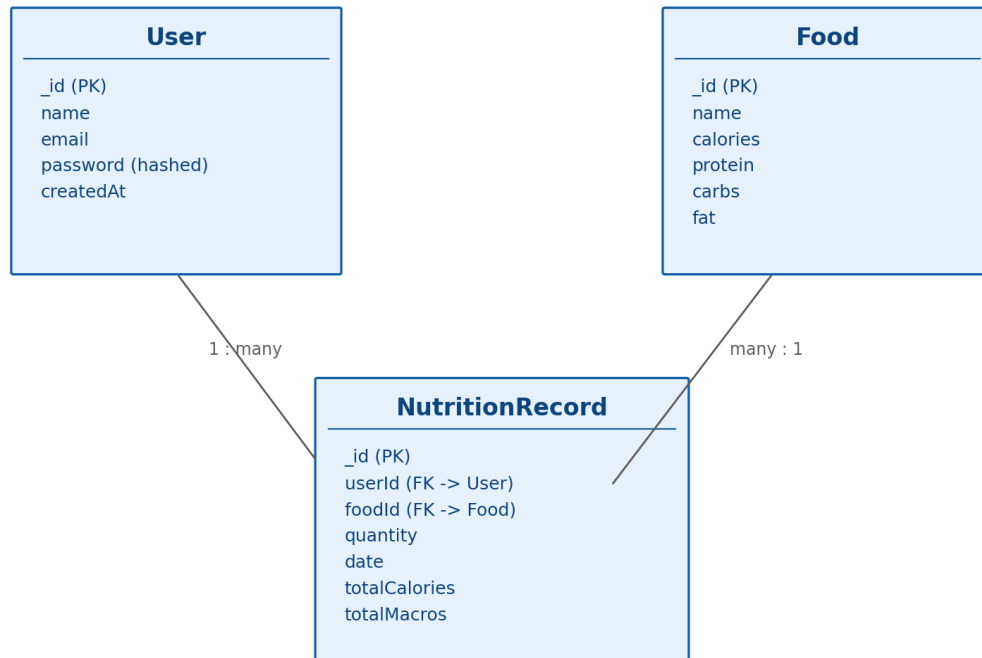
TECHNICAL ARCHITECTURE



In this architecture diagram:

- The frontend is represented by the “Frontend” section, including UI components such as the Landing page, Registration, Login, Dashboard, Food Search, and Calorie/Macronutrient views.
- The backend is represented by the “Backend” section, consisting of API endpoints for Authentication, Users, Food Search, and Dashboard data.
- The Database section represents MongoDB Atlas, which stores collections for Users, Food, and Nutrition Records.

ER-Diagram



The Nutrition Assistant ER diagram acts as the structural foundation of the system, showing how the User, Food, and NutritionRecord entities relate. Each User is uniquely identified and can create many NutritionRecord entries. Each NutritionRecord links a User to a specific Food item, along with the logged quantity, date, and calculated totals.

The relationships between these entities capture how the system works day to day: a single user can log many nutrition records over time, and a single food item can appear in many different users' records. This interconnected structure lets the system calculate accurate, real-time calorie and macronutrient totals for every user.

Key Features of Nutrition Assistant

User Registration & Authentication

- Secure sign-up and login process with a personalized dashboard.

Food Search

- Search thousands of food items and view nutritional information instantly.

Calorie Tracking

- Log daily food intake and monitor total calories consumed against personal goals.

Macronutrient Analysis

- Track protein, carbohydrates, and fat with detailed nutritional insights.

Dashboard

- View daily nutrition summary, calories consumed, and macronutrient distribution at a glance.

User-Friendly Interface

- Easy navigation, responsive UI, modern design, mobile-friendly layout.

ROLES AND RESPONSIBILITIES

1. User

- Register and create an account.
- Log in securely and access the personal nutrition dashboard.
- Search food items and view their nutritional information.
- Log daily food intake and track calorie/macronutrient totals.
- Review nutrition progress over time.

2. System (Nutrition Assistant Platform)

- Provide accurate, searchable food and nutrition data.
- Ensure secure registration, login, and user data handling.
- Record and recalculate nutrition totals after every logged food item.
- Maintain consistency and accuracy of dashboard summaries.
- Deliver a smooth, fast, and user-friendly interface.

User Flow



1. Entry Point

- User visits the Nutrition Assistant web app.

2. Authentication

- New User → Click Register → Enter details (name, email, password) → Account created → Redirect to Login.
- Existing User → Click Login → Enter credentials → Redirect to Dashboard.

3. Dashboard (Main Hub)

- View today's calorie total, macronutrient breakdown, and recent activity.

4. Browse & Search Food

- Search for food items and view calories, protein, carbohydrates, and fat.

5. Log Daily Intake

- Select a food item → enter quantity/serving size → confirm → totals update instantly.

6. Track Progress

- View updated calorie and macronutrient totals reflected on the dashboard.

7. Review Nutrition Summary

- Review daily nutrition summary and overall progress.

8. Logout

- End session securely.

MVC PATTERN

The Nutrition Assistant application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different

HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability: New features can be added easily by creating new routes, controllers, and models.
- Reusability: Logic in controllers and models can be reused across multiple parts of the application.
- Testing: Each layer can be tested independently, especially the controllers and models.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.

Project Setup and Configuration

Creating the project folder

1. Create a new folder for the project. 2. Inside that folder, create two new folders. 3. Name one client. 4. Name the other server. 5. Open the folder in VS Code.

Client setup (installing the React app)

- Open the client folder in the VS Code terminal.
`npm create vite@latest . -- --template react`
- Select React as the framework, and JavaScript as the variant.
`cd client npm install npm run dev`

Server setup (npm init)

- Open the server folder in the terminal.
`npm init -y`
- Create the entry file index.js and the folders: models, controllers, routes, middlewares, config.

BACKEND DEVELOPMENT

A representative backend structure for Nutrition Assistant, following standard MERN/MVC conventions consistent with the project's documented architecture (Users, Food, and Nutrition Records collections):

config

- db.js → Handles MongoDB Atlas connection setup and configuration.

controllers

- `authController.js` → Handles registration, login, and JWT issuance.
- `foodController.js` → Handles food search and nutrition data lookups.
- `nutritionController.js` → Manages logging of food intake and calorie/macronutrient calculations.
- `dashboardController.js` → Aggregates data for the dashboard summary view.

middlewares

- `authMiddleware.js` → Provides JWT authentication/authorization checks for protected routes.

models

- `userModel.js` → Defines the schema for user accounts and credentials.
- `foodModel.js` → Defines the schema for food items and their nutritional values.
- `nutritionRecordModel.js` → Defines the schema for logged food intake per user.

routes

- `authRoute.js` → API endpoints for registration and login.
- `foodRoute.js` → API endpoints for searching food and nutrition data.
- `nutritionRoute.js` → API endpoints for logging intake and fetching dashboard totals.

root files

- `index.js` → Main entry point of the server; sets up Express, routes, and middleware.
- `package.json` → Lists project dependencies, scripts, and metadata.

Note: file names above follow the same conventions used elsewhere in the project's documented architecture; confirm exact file names against your own server/folder before publishing this document.

DATABASE DEVELOPMENT

1. Configure MongoDB

- Install Mongoose in the server project directory:
`npm install mongoose`
- Create the database connection using the MongoDB Atlas URI.
- Create schemas using the entities from the ER diagram above.

Schemas

- 1) User Schema: represents user data and includes fields such as name, email, and password (hashed).
- 2) Food Schema: represents food data such as name, calories, protein, carbohydrates, and fat.

3) NutritionRecord Schema: represents a logged food entry, including user reference, food reference, quantity, date, and calculated totals.

FRONTEND DEVELOPMENT

A representative frontend structure for Nutrition Assistant, matching the pages and components implied by the project's key features (Registration, Login, Dashboard, Food Search, Calorie Tracker, Macronutrient view):

client/src/components

- axiosInstance.js → Pre-configured Axios instance for making API calls.
- Navbar.jsx → Navigation bar UI component.
- Login.jsx → Login form component for user authentication.
- Register.jsx → Registration form component for new users.

client/src/context

- AuthContext.jsx → Provides global authentication/user state across the app.

client/src/pages

- Landing.jsx → Public landing page shown before login.
- Dashboard.jsx → Main dashboard showing nutrition summary.
- FoodSearch.jsx → Food search page with nutrition lookup.
- CalorieTracker.jsx → Page for logging daily food intake and viewing calorie totals.
- MacronutrientDashboard.jsx → Displays protein/carb/fat breakdown.

Note: as with the backend, these file names follow standard conventions consistent with the project's feature set; adjust to match your actual client/src folder.

Steps for Project Execution

Step 1: Set Up the Frontend (React App)

a) Open a terminal and navigate into the client folder:

```
cd client
```

b) Install dependencies and start the development server:

```
npm install npm run dev
```

c) The app should now be running on:

```
http://localhost:5173
```

Step 2: Set Up the Backend (Express Server)

a) Open a new terminal tab/window.

b) Navigate into the server folder:

```
cd server
```

Step 3: Configure Environment Variables

- Inside the server folder, create a .env file.
- Add the following variables (matching the project's .env.example):

```
PORT=5000 MONGODB_URI=Your_MongoDB_URI JWT_SECRET=Your_JWT_Secret  
API_KEY=Your_API_Key
```

Step 4: Start the Backend Server

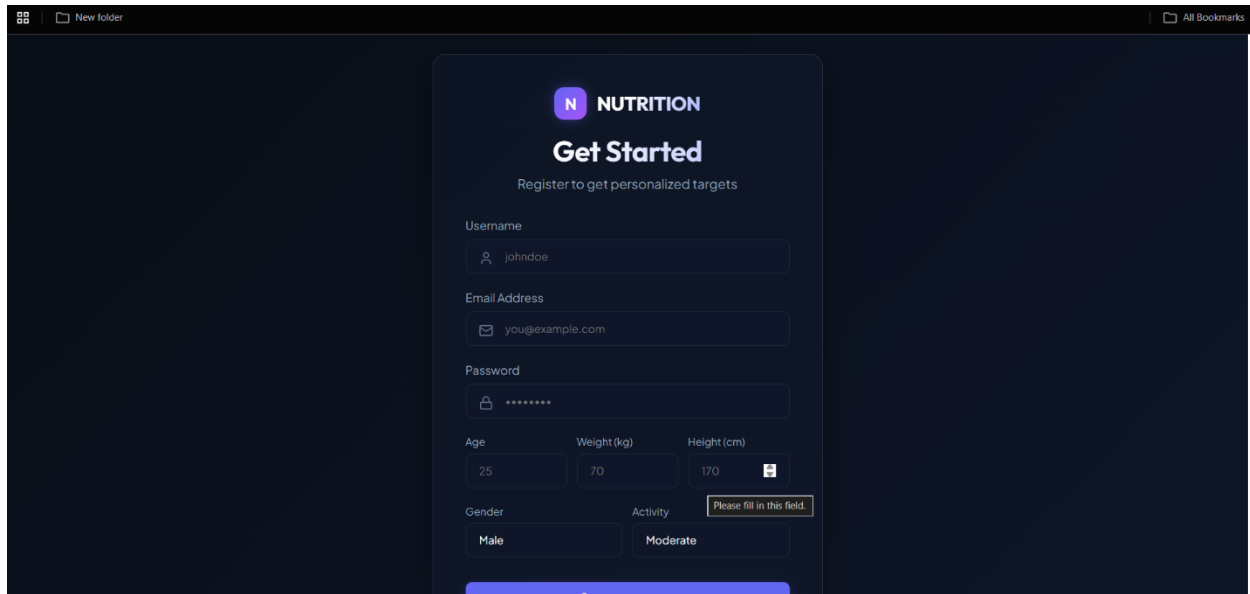
Install dependencies and run the backend server:

```
npm install npm start
```

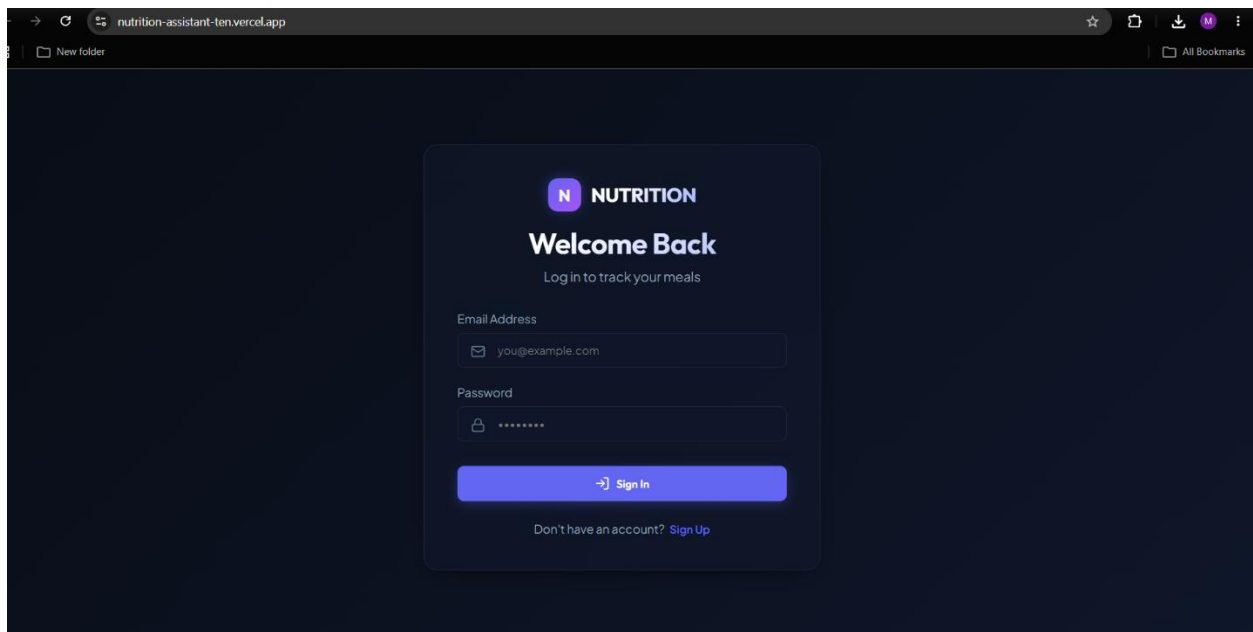
The server should start on:

```
http://localhost:5000
```

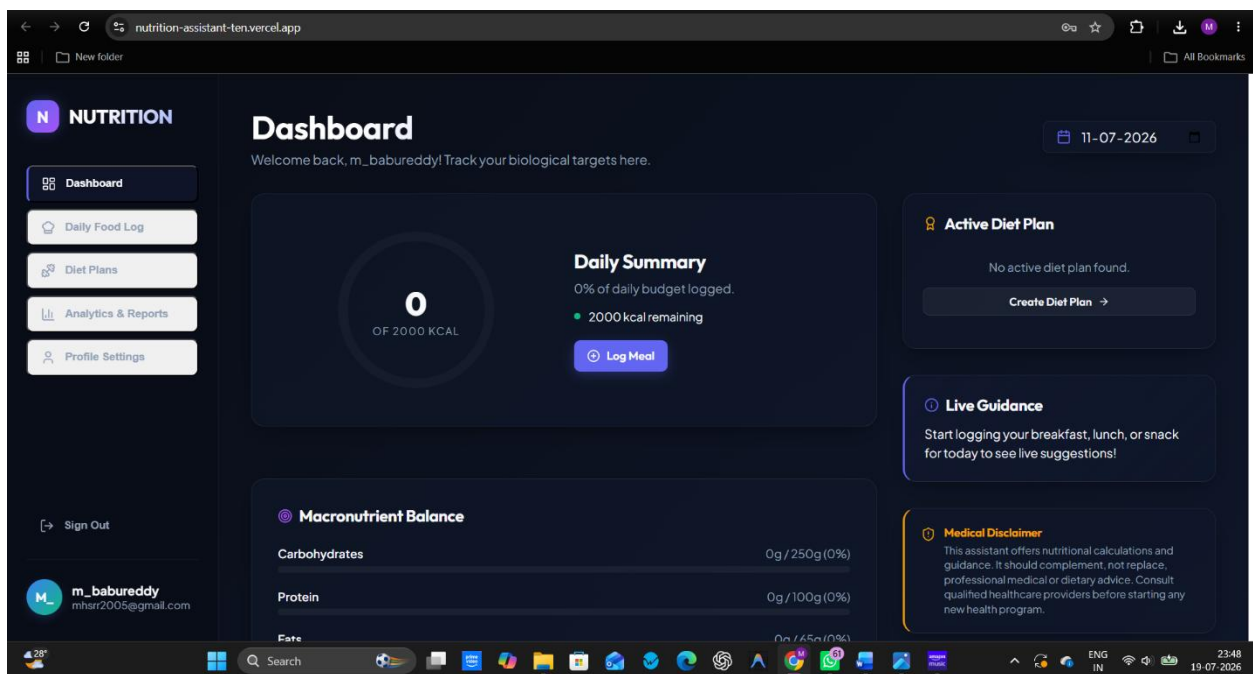
Output Screenshots

A screenshot of a web browser displaying the 'Get Started' registration page for a nutrition app. The page has a dark blue background. At the top, there's a logo with a purple 'N' and the word 'NUTRITION'. Below the logo, the heading 'Get Started' is followed by the subtext 'Register to get personalized targets'. The registration form includes fields for 'Username' (with 'johndoe' entered), 'Email Address' (with 'you@example.com' entered), and 'Password' (with '*****' entered). Below these are three input fields for 'Age' (25), 'Weight (kg)' (70), and 'Height (cm)' (170). At the bottom, there are two buttons for 'Gender' ('Male') and 'Activity' ('Moderate'). A small yellow warning box says 'Please fill in this field.' next to the Activity button. A purple 'Get Started' button is at the very bottom of the form.

Registration Page



Login Page



Dashboard

Links

Source Code Link: <https://github.com/IslavathuSurya-89/Nutrition-Assistant>

Live Demo Link: <https://nutrition-assistant-ten.vercel.app/>

Project Demo Video: <https://drive.google.com/file/d/1qSYxZaZ1YlfyhAmkOxF3N0Kg1IWGIVK6/view>

